

Somaiya Vidyavihar University
K J Somaiya School of Engineering
Department of Electronics and Telecommunication Engineering



SOMAIYA
VIDYAVIHAR UNIVERSITY
K. J. Somaiya School of Engineering
(formerly K. J. Somaiya College of Engineering)

Somaiya Vidyavihar University
K J Somaiya School of Engineering

Department of Electronics and Telecommunication Engineering



Batch: A2 Roll No.: 16010324044

Experiment / assignment / tutorial No. 6

Experiment No: 6

Title: Queries based on Triggers

Objective: To be able to use trigger on table.

Expected Outcome of Experiment:

CO 3 : Use SQL for Relational database creation, maintenance and query processing

Books/ Journals/ Websites referred:

1. Dr. P.S. Deshpande, SQL and PL/SQL for Oracle 10g. Black book, Dreamtech Press
2. www.db-book.com
3. Korth, Sliberchatz, Sudarshan : "Database Systems Concept", 5th Edition , McGraw Hill
4. Elmasri and Navathe, "Fundamentals of database Systems", 4th Edition, PEARSON Education.

Resources used: Postgresql

Theory

Triggers are database call-back functions, which are automatically performed/invoked when a specified database event occurs.

Triggers can be specified to fire

- Before the operation is attempted on a row (before constraints are checked and the INSERT, UPDATE or DELETE is attempted)
- After the operation has completed (after constraints are checked and the INSERT, UPDATE, or DELETE has completed)
- Instead of the operation (in the case of inserts, updates or deletes on a view)

Department of Electronics and Telecommunication Engineering
DMSL –Sem- VI- Jan –April 2026

Somaiya Vidyavihar University
K J Somaiya School of Engineering
Department of Electronics and Telecommunication Engineering

Experiment No: 6

Title: Queries based on Triggers

Objective: To be able to use trigger on table.

Expected Outcome of Experiment:

CO 3 : Use SQL for Relational database creation, maintenance and query processing

Books/ Journals/ Websites referred:

1. Dr. P.S. Deshpande, SQL and PL/SQL for Oracle 10g.Black book, Dreamtech Press
2. www.db-book.com
3. Korth, Silberchatz, Sudarshan : “Database Systems Concept”, 5th Edition , McGraw Hill
4. Elmasri and Navathe,”Fundamentals of database Systems”, 4th Edition,PEARSON Education.

Resources used: Postgresql

Theory

Triggers are database call-back functions, which are automatically performed/invoked when a specified database event occurs.

Triggers can be specified to fire

- Before the operation is attempted on a row (before constraints are checked and the INSERT, UPDATE or DELETE is attempted)
- After the operation has completed (after constraints are checked and the INSERT, UPDATE, or DELETE has completed)
- Instead of the operation (in the case of inserts, updates or deletes on a view)

The basic syntax of creating a trigger is as follows –

CREATE TRIGGER trigger_name [BEFORE|AFTER|INSTEAD OF] event_name

Somaiya Vidyavihar University
K J Somaiya School of Engineering
Department of Electronics and Telecommunication Engineering

ON table_name

[

-- Trigger logic goes here....

];

event_name could be INSERT, DELETE, UPDATE, and TRUNCATE database operation on the mentioned table table_name. You can optionally specify FOR EACH ROW after table name.

The following is the syntax of creating a trigger on an UPDATE operation on one or more specified columns of a table as follows –

```
CREATE TRIGGER trigger_name [BEFORE|AFTER] UPDATE OF column_name
```

```
ON table_name
```

```
[
```

```
-- Trigger logic goes here....
```

```
];
```

Problem Statement: Library Book Issue and Fine Management System

A public library wants to automate the tracking of book issues, returns, and fine calculations without depending on the front-end application to enforce these rules. The library maintains records of books, members, and issue transactions (book borrowed, date issued, due date, date returned). Currently, staff manually update book availability, calculate overdue fines, and maintain audit logs of changes — a process that is slow and prone to errors. The management wants the database itself to automatically respond to key events: whenever a book is issued, its available copies should be reduced; whenever a book is returned, its available copies should be restored and a fine calculated if returned late; and every insert, update, or delete on the transaction table should be logged for audit purposes. Additionally, the system should prevent invalid operations — such as issuing a book that has zero copies available — before the operation is even attempted.

You are asked to design the database schema and write triggers using **BEFORE**, **AFTER**, and **INSTEAD OF** clauses on **INSERT**, **UPDATE**, and **DELETE** events to meet the following requirements:

Somaiya Vidyavihar University
 K J Somaiya School of Engineering
Department of Electronics and Telecommunication Engineering

1. **BEFORE INSERT trigger** on the `Issue` table — prevent a book from being issued if `available_copies = 0` for that book (raise an exception and abort the insert).
2. **AFTER INSERT trigger** on the `Issue` table — automatically decrement `available_copies` in the `Book` table by 1 each time a book is issued.
3. **AFTER UPDATE trigger** on the `Issue` table — when `return_date` is updated (book returned), automatically increment `available_copies` back by 1, and calculate and insert a fine into a `Fine` table if `return_date > due_date` (e.g., ₹5 per day late).
4. **AFTER DELETE trigger** on the `Issue` table — log the deleted transaction (member, book, dates) into an `Audit_Log` table with a timestamp and the type of operation performed.
5. **Row-counting trigger** — write a trigger that counts and records the number of new tuples inserted into the `Issue` table by each `INSERT` statement (single or bulk insert) into a separate tracking table.
6. **INSTEAD OF trigger** on a view (e.g., `Active_Issues_View`) that shows currently unreturned books — so that inserting/updating through the view correctly redirects the operation to the base `Issue` table.

Implementation Screenshots (Problem Statement, Query and Screenshots of Results):

Create the database tables

	<code>log_id</code> [PK] integer	<code>inserted_count</code> integer	<code>insertion_time</code> timestamp without time zone
--	-------------------------------------	--	--

Insert sample data

	<code>member_id</code> [PK] integer	<code>member_name</code> character varying (100)
1	1	Rahul
2	2	Amit
3	3	Priya

BEFORE INSERT trigger

	<code>issue_id</code> [PK] integer	<code>member_id</code> integer	<code>book_id</code> integer	<code>issue_date</code> date	<code>due_date</code> date	<code>return_date</code> date
1	2	1	3	2026-09-16	2026-09-23	[null]

AFTER INSERT trigger

	<code>issue_id</code> [PK] integer	<code>member_id</code> integer	<code>book_id</code> integer	<code>issue_date</code> date	<code>due_date</code> date	<code>return_date</code> date
--	---------------------------------------	-----------------------------------	---------------------------------	---------------------------------	-------------------------------	----------------------------------

Somaiya Vidyavihar University
K J Somaiya School of Engineering
Department of Electronics and Telecommunication Engineering

AFTER UPDATE trigger (Returning)

	fine_id [PK] integer	issue_id integer	fine_amount numeric (10,2)	fine_date timestamp without time zone
--	--------------------------------	----------------------------	--------------------------------------	---

AFTER DELETE trigger

	audit_id [PK] integer	issue_id integer	member_id integer	book_id integer	issue_date date	due_date date	return_date date	operation_type character varying (20)	operation_time timestamp without time zone
--	---------------------------------	----------------------------	-----------------------------	---------------------------	---------------------------	-------------------------	----------------------------	---	--

Row-counting trigger

	log_id [PK] integer	inserted_count integer	insertion_time timestamp without time zone
1	1	2	2026-09-16 15:28:24.591472

Create Active Issues View

	issue_id integer	member_id integer	book_id integer	issue_date date	due_date date	return_date date
1	2	1	3	2026-09-16	2026-09-23	[null]
2	3	1	1	2026-09-16	2026-09-23	[null]
3	4	2	2	2026-09-16	2026-09-23	[null]

INSTEAD OF INSERT trigger

	issue_id integer	member_id integer	book_id integer	issue_date date	due_date date	return_date date
1	2	1	3	2026-09-16	2026-09-23	[null]
2	3	1	1	2026-09-16	2026-09-23	[null]
3	4	2	2	2026-09-16	2026-09-23	[null]
4	5	3	1	2026-09-16	2026-09-23	[null]

INSTEAD OF UPDATE trigger

	issue_id [PK] integer	member_id integer	book_id integer	issue_date date	due_date date	return_date date
1	3	1	1	2026-09-16	2026-09-23	[null]
2	4	2	2	2026-09-16	2026-09-23	[null]
3	5	3	1	2026-09-16	2026-09-23	[null]
4	2	1	3	2026-09-16	2026-09-25	[null]

Somaiya Vidyavihar University
K J Somaiya School of Engineering
Department of Electronics and Telecommunication Engineering

Post Lab Questions:

- 1. Write a trigger to count the number of new tuples inserted using each insert statement.**

```
SQL
CREATE OR REPLACE FUNCTION count_inserted_rows()s
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    row_count INT;
BEGIN
    SELECT COUNT(*) INTO row_count
    FROM new_rows;

    INSERT INTO Insert_Count_Log(inserted_count)
    VALUES(row_count);

    RETURN NULL;
END;
$$;

CREATE TRIGGER count_issue_inserts
AFTER INSERT ON Issue
```

Somaia Vidyavihar University
K J Somaia School of Engineering
Department of Electronics and Telecommunication Engineering

```
REFERENCING NEW TABLE AS new_rows  
  
FOR EACH STATEMENT  
  
EXECUTE FUNCTION count_inserted_rows();
```

2. Trigger is special type of _____ procedure.

a) Stored

b) Function

c) View

d) Table

3. Triggers can be enabled or disabled with the _____ statement.

a) ALTER TABLE statement

b) DROP TABLE statement

c) DELETE TABLE statement

d) None of the mentioned

Conclusion:

In this experiment, I successfully performed using PostgreSQL triggers. BEFORE, AFTER and INSTEAD OF triggers were implemented to automate book availability, book returns, fine calculation, audit logging and operations through a view. A statement-level trigger was also used to count inserted tuples. Thus, database triggers can automatically enforce rules and maintain data consistency without relying entirely on the front-end application.

Signature of faculty in-charge

**Department of Electronics and Telecommunication Engineering
DMSL –Sem- VI- Jan –April 2026**